The Definitive Guide to Technical Debt 🦠

How to prevent it, measure it, and eventually pay it back. Backed by deep research and tens of stories from the community.



LUCA ROSSI

APR 24, 2024 · PAID

29 4 Share

ARTICLE VOICEOVER

1× 0:00 -21:11

Refactoring has ~75K subscribers today, who include people with very different jobs: engineers, managers, CTOs, founders, and more.

Whenever I write an article, it is naturally *more* or *less* interesting to subsets of these people. When I wrote about the tech talent landscape, it especially resonated with those into hiring. When I wrote The Startup Healthcheck, it was especially useful to founders.

But there are also… exceptions: topics that everybody understands and have lived through in a way or another.

The mother of all these topics is **technical debt** 🦠

I have written twice about it: on the first time, it was to define it, while on the second one it was to keep it under control.

Both articles are more than two years old now. In the meantime, I have learned a lot by speaking with hundreds of managers and engineers — through the newsletter, and especially through the community, which grew from 50 to more than 750 members.

So, today I am publishing a full, thorough guide on how to handle technical debt: how to prevent it, measure it, and ultimately pay it back.

I write one of such **Guides** every month — this will be the April one. Guides are longer, more in-depth and more researched articles than usual, that act as *primers* on a given topic.



Refactoring Guides 📖
LUCA ROSSI · JULY 31, 2023
Read full story →

This guide will also be **extremely practical**. I want you to leave this article thinking: "*ok, now I can do this and that*".

So, here is our agenda for today:

1. 📖 **What is Technical Debt** — definitions first, to get everybody on the same page.
2. ⚖️ **Good vs Bad Debt?** — is there even such a thing like good debt?
3. 🔭 **How to Prevent Debt** — daily practices to keep code clean and simple
4. 💸 **How to Repay Debt** — what to do when the crap is already there.
5. 📏 **How to Measure Debt** — how to know how *bad* things are, and how *better* they could be.
6. 📣 **How to Advocate for Debt** — how to convincingly tell others, especially non-engineers.
7. 💬 **Community thread** — giant thread with 25+ replies about real stories of how tech leaders in the Refactoring community handle tech debt.

Let's dive in!

📖 What is Technical Debt

Ward Cunnigham is one of the signers of the original Agile manifesto, and the *inventor* of the technical debt metaphor.

Here is the sentence where it is believed that this metaphor was used for the first time:

> *If we fail to make the program aligned with what we understand to be the proper way to think about our [...] objects, then we are going to continuously stumble*

> *into disagreement, and that would slow us down like paying interest on a loan.*

Ward describes debt as the natural result of writing code about something we don't have a **proper understanding** of.

He doesn't talk about poor code — which, he says, accounts for a very minor share of debt — he talks about **disagreement** between business needs and how the software has been written.

In other words, software is a **formal description** of some business reality and needs. Whenever such a description doesn't match reality, we get tech debt:

- A design that doesn't scale properly fails to take into account the reality of business volume.
- A leaky abstraction is like a bad rule that doesn't fit reality anymore, and needs continuous exceptions to keep working.

So, how did we get there? How do we land in disagreement?

⚖️ Good vs Bad debt

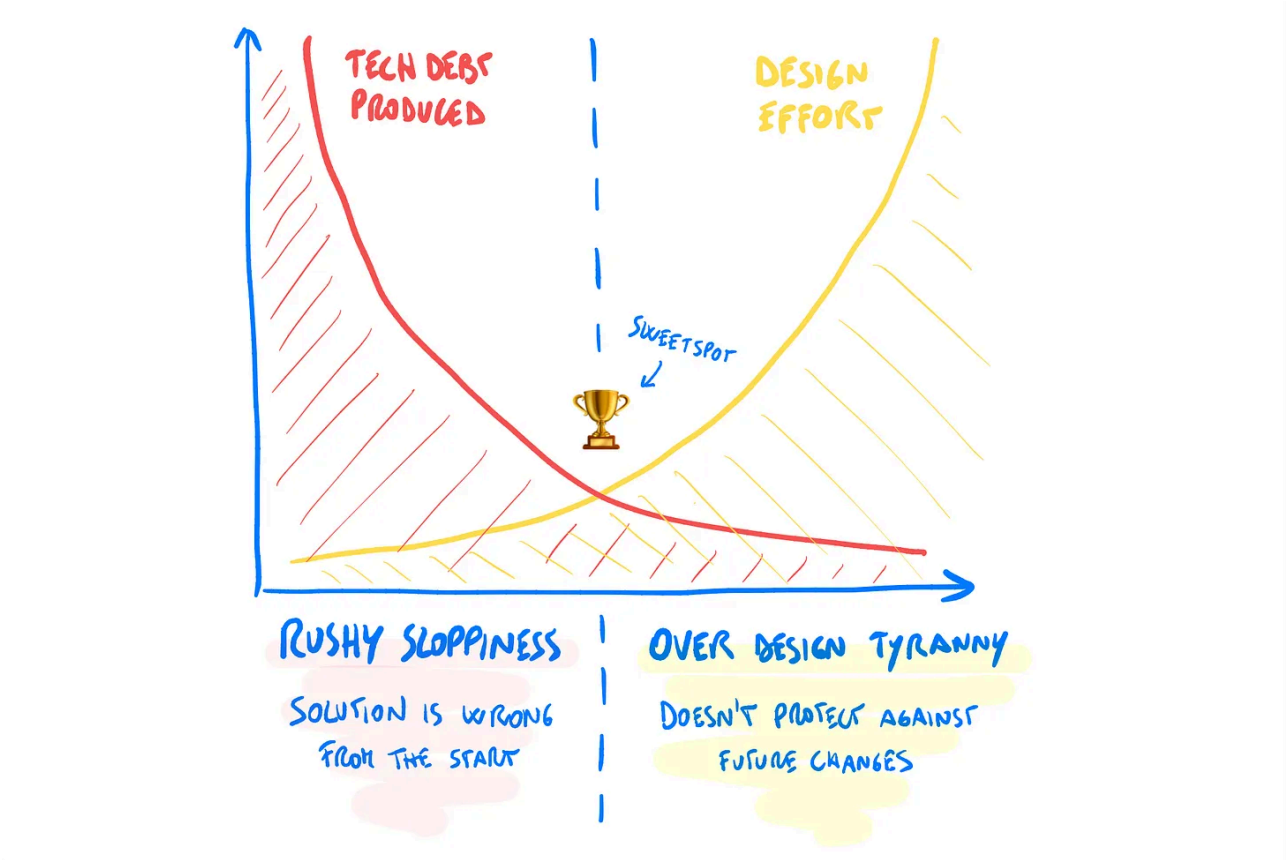
Whenever you find that your software doesn't fit reality, it is useful to ask yourself: *when did it happen*?

There are two macro answers to this:

- **From the start** — engineering, product, and business simply didn't fully understand one another, and built a (partially) wrong thing.
- **Over time** — reality gradually *changed* into something else and invalidated some assumptions.

These differences are not clear cut, as some future changes *can* and should be predicted from the start, but you can still get a sense of where the problem is based on **how early and frequent** the rework is.

Is it only *weeks* after release, or months, or years? What does that **code churn** look like? Is it the first time you are touching it or is it a regular?



At this point you may ask: since reality changes anyway, isn't debt inevitable? Isn't some debt even *good* when it allows us to go faster, granted we keep it under control?

This question is tricky and the answer goes back to the knowns and unknowns model. When you design software, you have three sets of *knowns*:

1. 🟢 **Known knowns** — requirements you know and are under control of.
2. 🟡 **Known unknowns** — uncertain items you are at least aware of, like scale, or some future product directions.
3. 🔴 **Unknown unknowns** — all the unforeseeable ways things can fail.

You can classify bad, good, and best designs based on how they account for these:

- 🥉 **Bad design** • doesn't even account for (1). Engineering didn't really understand product, made a set of wrong assumptions, and design was wrong from the start.
- 🥈 **Good design** • solves for (1), but doesn't account for (2). E.g. if scale is an unknown, failure means to commit decisively to a solution that is opinionated

about it — you either invest a lot on premature (and speculative) scaling, or on something that will never scale and it is hard to undo.

- 🥇 **Best design** • solves for (1) and accounts for (2). For example, it quickly pushes out a design that *doesn't* scale but it is easy to scrap later, once you have figured things out.

So, back to good vs bad debt, and debt as disagreement:

- **Bad debt is *unaware* disagreement** — you didn't account for things that were under your control.
- **Good debt is *delayed* disagreement** — you shrink the scope of the agreement today, and accept some disagreement later.

This angle is similar to Martin Fowler's quadrant (it's arguably a subset of it).

📈 Tech debt vs Product stage

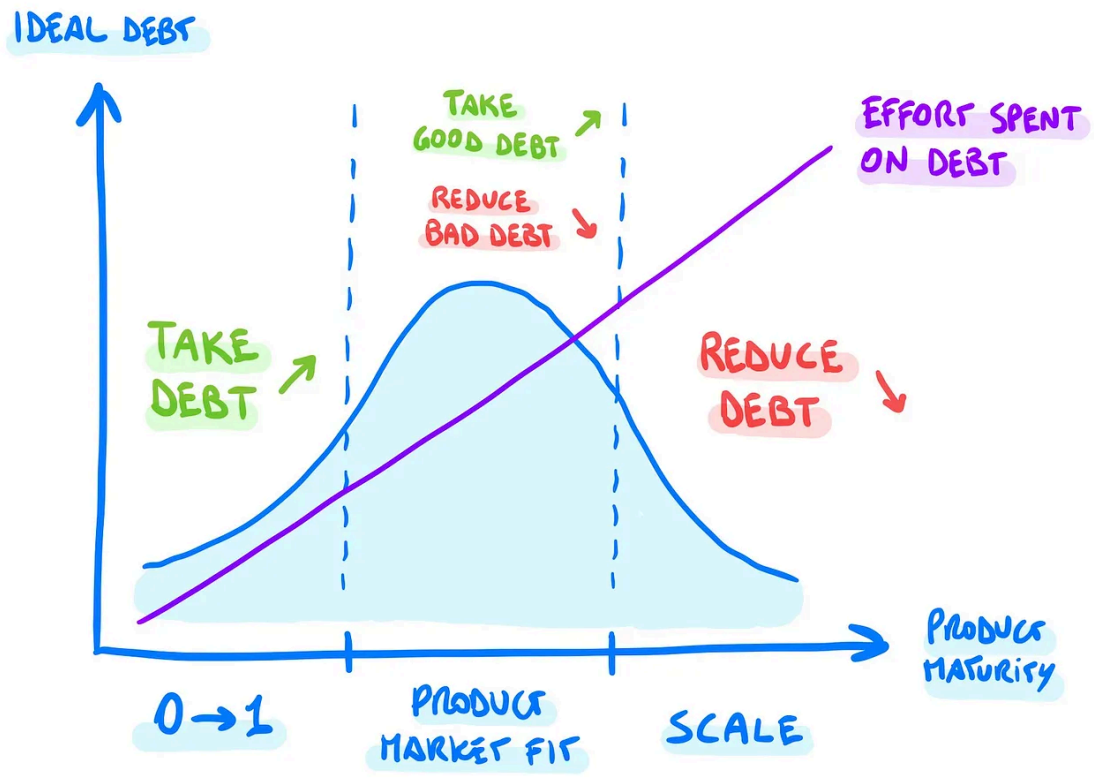
I believe that the opportunity to *accept* the tradeoff of creating intentional debt depends on your product stage.

Fast growth *naturally* leads to debt, because when the product changes at a fast rate, your engineering abstractions get invalidated equally fast. Such volatility changes with the maturity of your product.

For the sake of simplicity, let's consider three stages:

- **0 to 1** — you start building a product from scratch, with a set of initial assumptions.
- **Product Market Fit** — you figure out what works, double down on it, and scrap the rest.
- **Scale** — you grow your business predictably and harden your tech.

The earlier you are on this scale, the more your product needs to move fast, and the more *leverage* you get by accruing debt.



The later you are on the scale, the more debt becomes a drag that *prevents* your product from growing. Your scale is such that you get the most leverage by *repaying* debt.

🔭 How to prevent tech debt

So far, this was pretty much theory! How do you prevent tech debt *in practice*? Here are my favorite recipes:

1) Write code like a toddler 👶

I borrow this analogy from James Cowling, CTO at Convex and former Principal Engineer at Dropbox.

I interviewed James a few months back and we agreed that the best code we ever saw was remarkably simple. Simple means many things:

- **Few abstractions** — we don't over commit on ideas that may backfire later. Rather than DRY, use the Rule of Three: only create an abstraction when there

are three instances of the same thing.

- **Easy to refactor** — I like to think about good code like those sailor knots that should be strong enough to hold for a while, but easy to dismantle when not needed anymore.
- **Easy to understand** — if tech debt is misunderstanding, then clear code goes a long way to prevent it. We could spend several hours debating what it means to write clear code, but it often involves doing *unclever* things, like using good names, or adding comments.

2) Create healthy defaults 🩺

Let your stack do the work for you. There is a lot you can do to (semi) automatically prevent a long tail of small tech debt:

- **Invest in static analysis** — plenty of issues can be intercepted by good static analysis, and the scope is only increasing also thanks to AI: style, security, coverage, unused code, and more. This is a total no-brainer today, and you have a long list of tools to choose from. At Refactoring we have long been fans of Codacy, whose team also hangs regularly in the community (no affiliation!).

> *Static analysis isn't the be-all and end-all of tech debt, by any stretch, but, for example, I've been working on our VSCode IDE extension myself lately and the SA rules we have configured ensure that, for instance, I can't merge to main without making sure my import statements are tidied up and that I don't have unused variables in code.*
>
> *These little smells over time degrade the code to an unreadeable mess so ensuring they aren't introduced in the first place is a very easy way of reducing the quantity of tech debt coming in.*
>
> *— Kendrick Curtis, VP of Engineering at Codacy*

- **Use a static-typed language** — this is bound to be controversial but, in my experience, static (or gradual) typing vastly improves the maintainability of any non-trivial code base. And if you think it's too late, think again: Stripe migrated 3.7M lines of code to Typescript in one PR.

- **Invest in test coverage** — a good level of coverage is the first line of defense against small-scale debt. Leaky abstractions around edge cases, sneaky changes of contract, can be all intercepted with a good test suite. I wrote a full guide about it last year.

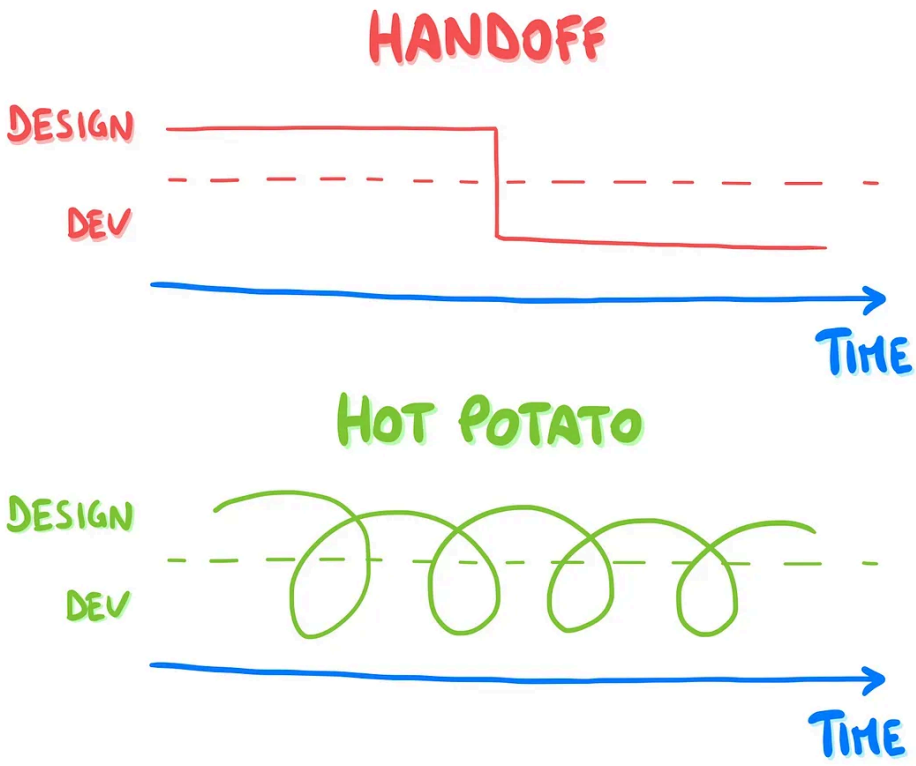
3) Invest on good collaboration 🤝

If tech debt is disagreement, you can prevent a lot of it by simply working better as a team.

Code reviews are there for a reason but, in my experience, they are rarely enough. They are often superficial and hasty, and they get summoned when the bulk of the work has been done already.

So, collaboration benefits, like many other things, from the **shift-left approach** — *do it more and do it earlier*:

- 👯 **Pair programming** — in coding, this means pair programming: when you work together with a peer, you are guaranteed to produce a better design, and write less code because of it.
- 🥔 **Hot potato** — before you even get to coding, it means having engineers sit with product and design people to figure out things together, and having the necessary *back and forths* to make things right. This is the *hot potato* process, as opposed to the dreaded *handoff* 👇



> *I went with the "prevention is the best cure" approach. Mostly that meant holding myself and the other contributors to a higher standard than we were used to. I took time to sit down 1:1 with the team members to walk through PRs, explaining why we should do things differently etc.*
>
> — Clemens Adolphs, COO at vortX

🏦 How to repay tech debt

Enough with the prevention — what if debt is already there? Watching how great teams operate on this, a few themes stay remarkably constant. Processes to reduce debt are always:

1. 🎯 **Intentional** — managers can articulate well what they do to keep debt in check. Everyone on the team knows how this works.
2. 〰️ **Continuous** — processes are never based on isolated, sparse initiatives. People address debt every day.
3. 🎨 **Diverse** — debt is managed in multiple ways, based on its type and size. There is no one-size-fits-all process.

Let's see what processes you can setup based on the size of the debt itself.

1) Small Debt 🟩

Small debt is about minor improvements you can make in a few hours of time. Examples are

- *Adding missing tests*
- *Deduplicating some code with a simple abstraction*
- *Adjusting a piece of code to make it adhere to your codebase conventions*

The best way to address small debt is to make these changes whenever you are working on that code for any other reason. You basically follow Robert Martin's boy scout rule:

Always leave the code better than you found it.

This is effective, as opposed to scheduling an independent task, because it **avoids context-switch** and makes the change cheaper. However, it also means the timeline for such improvements is *best-effort* (it depends on other tasks), so it is only suitable for issues that are not too harmful.

2) Medium Debt 🟨

Medium debt is about issues you need to prioritize and schedule, because they are somewhat impactful, but you can still fix them in between a few hours and a couple of days.

Examples are:

- *Automating a tedious manual process*
- *Adjusting performance of a small service that's been degrading over time*
- *Removing a bottleneck in the CI/CD pipeline to shave a few minutes off the deploy time*

How should you address these?

In my experience, the best approach is to allocate some **fixed time** to them. This may happen in various ways:

- ⏱️ **Regular time** — You may have the *whole* team spend some share of their time (e.g. 20-30%) on maintenance. At my startup we had the *Friday Bugs,* where we used Fridays to catch up on bug fixing and other work under the hood.
- 🧑‍💻 **Individual rotation** — You may have people rotate on maintenance, possibly combining it with on-call duties (e.g. see the Bug Duty).
- 🎽 **Team rotation** — You may have full weeks / cycles dedicated to maintenance.

I wrote more about how to handle maintenance in this previous piece 👇



How to Plan for Maintenance 🛠️
LUCA ROSSI · APRIL 13, 2023
Read full story →

This is not the only possible approach. There are teams who put these tasks in a backlog together with other product stuff, prioritize the ones against the others and plan what to do cycle by cycle. While this is not wrong by itself, it has a few drawbacks:

- **Expensive** — negotiating tech debt against product work every sprint can bring serious overhead, especially for minor tasks.
- **Apples vs oranges** — sometimes it's hard to identify the ROI of small maintenance items.
- **Unreliable** — if you have to negotiate it every time, it is more likely that maintenance will be left behind when you are under pressure, and more generally it will be harder to keep a consistent pace.

Nevertheless, maintenance items most likely benefit from a backlog of their own. It should just be separate from the rest 👇

> *The engineering team maintains a tech debt roadmap, evaluating it monthly.*

> *This roadmap typically spans a few months into the future, featuring one prominent task currently in progress—accompanied by a non-technical explanation of its importance—and several upcoming tasks. If a task is significant enough to potentially impact business operations, we communicate this to the business development team, providing reasoning for its necessity.*
>
> *— Tomas Zaoral, EM at Purple Technology*

3) Large Debt 🟥

Large debt is those strategic initiatives that span between a few weeks and a few months of work.

Examples are:

- *Refactoring a large piece of legacy code to enable product improvements*
- *Migrate a service to a different library / framework because of some new requirements.*
- *Rework a piece of your infrastructure to improve reliability and performance.*

These need to be planned on a longer timeframe — e.g. quarterly — and evaluated strategically, just like regular product work, based on their **business opportunity**.

Calculating this opportunity is not easy, though, because such initiatives sometimes don't impact the business *directly*, but via **second-order effects** like team velocity, happiness, or reduced turnover.

So, how do you measure? 👇

📏 How to measure tech debt

There are several ways to measure how *bad* things are — or how *better* they could be — to make a solid case for tech debt initiatives.

1) Measure where your time goes 📊

You can track your *engineering investment* to figure out where your dev time really goes. For example, I am a fan of the balance framework, which divides work into four areas:

1. 🩺 **KTLO** — mandatory stuff that Keeps The Lights On.
2. 🔨 **New things** — work towards business goals, like new features.
3. 🔧 **Improvements** — work to improve existing features, including performance, reliability, etc.
4. ⚙️ **Productivity** — improvements to DX.

You can track this semi-automatically with tools like Swarmia or LinearB.



For teams that do this for the first time, this is often an eye-opener. It creates *alignment* with the business, builds trust, and counters natural biases your team may have towards specific types of work.

It is also great to back conversations about debt. Are you spending 50% of your time on KTLO? It's probably time for something strategic about it.

2) The Riot Model 🕹️

For measuring individual pieces of debt, I am a fan of the [system used by Riot](#).

They use three main metrics:

- 💣 **Impact** — business metrics impacted by the debt, or the value you unlock by repaying it. Crucial for the ROI equation.
- 💸 **Fix Cost** — a rough estimate based on some feasible solution. For the sake of prioritization, there is no need for it to be accurate. Simple t-shirt sizing (e.g. S, M, L, XL) is fine.
- 🦠 **Contagion** — this answers the question: "*if this debt is allowed to continue to exist, how much will it spread?*". It's a great angle because, in this regard, not all debt is created equal.

The contagion metric is particularly powerful because it informs on how impact and cost change over time. Bill Clark, former EM for League of Legends, explains this well:

> If a piece of tech debt is well-contained, the cost to fix it later compared to now is basically identical. You can weigh how much impact it has today when determining when a fix makes sense.
>
> *If, on the other hand, a piece of tech debt is **highly contagious**, it will steadily become harder and harder to fix. What's particularly gross about contagious tech debt is that its impact tends to increase as more and more systems become infected by the technical compromise at its core.*

3) Consider *enablers* 🚀

Finally, consider what repaying debt *enables* you to do. Often times, refactoring **unlocks product improvements**.

Sit down with other stakeholders and figure out opportunities around rework areas. Leaders from other areas may identify additional benefits and ultimately support you in advocating for this work.

📣 How to advocate for tech debt

So, all in all, how do you advocate for tech debt initiatives successfully?

I agree with my friend Joel when he says that the word *tech debt* is most often a bad framing for rework. It feels like you have to do them because of a *duty* (repay a debt) rather than a true business opportunity. So here is my advice around this:

1) Present a good ROI 📏

Measure both the *bad* (e.g. time spent regularly on maintenance, frustration, latency, …) and the *good* (e.g. product improvements *enabled* by the rework).

Do your homework and treat it like a product initiative, not like a debt.

2) Create success stories 🏆

Big reworks are always daunting, no matter how good your story is. To create enough momentum to go after them, create a solid **track record** of small-scale debt that has been successfully repaid, with proven outcomes.

Whenever you do some small rework, don't stop there: measure the outcome and make it visible. In other words, build trust 👇

3) Build trust ❤️

No matter how good you are, tech debt items will always be at a disadvantage when negotiated against regular product features.

That's because features have immediate impact — e.g. increasing the conversion rate at checkout — while refactoring has a **delayed feedback loop** that is measured in efficiency, velocity, and all kinds of second-order effects.

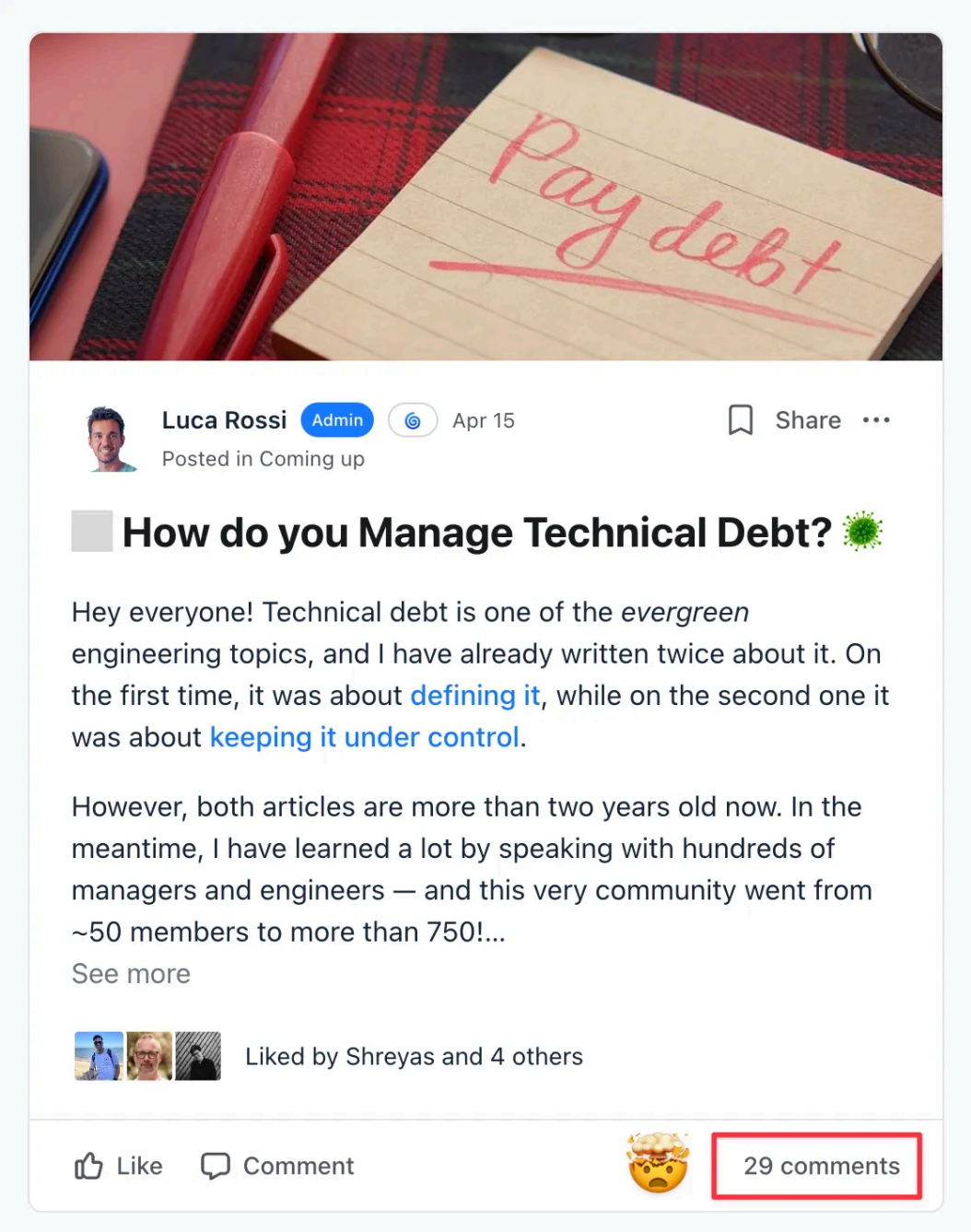
It's not a level playing field, so *trust* plays a decisive role. Many times, the #1 element that enables repaying debt is **strong relationships** between stakeholders, as opposed to precise KPIs.

This trust, however, is earned. You grow it over time by being rigorous and by creating a good track record. These, in turn, give you the room to go after bigger and bigger things.

💬 Community Thread

This article could never be possible without the insane amount of great ideas shared in the community.

We discussed this topic together and the thread got ~30 comments full of stories of managing debt in real life 👇



I usually embed these stories in the article (I did with some today) but this time there were too many, so I am attaching the full community thread below!

It's a first, so let me know if you like it! And if you haven't already, you can join the community at **this link**.

📚 Resources

Here are some interesting pieces about tech debt that helped me on this:

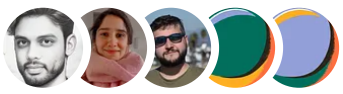
- 📑 **Technical debt is a terrible metaphor** — by Joel Chippindale. A thoughtful take on why we should stop talking about debt.
- 📑 **Tech Debt Financing** — by Jack Danger. As opposed to Joel, Jack goes all-in on the financial metaphor, introducing more elements that help you prioritize and repay debt.
- 📑 **Long Term Refactors** — by Max Chernyak. Max focuses on how to handle big refactorings, which are always the most challenging pieces of debt to advocate for and eventually execute.
- 📑 **Technical Debt Quadrant** — by Martin Fowler. Evergreen piece that classifies debt in *reckless* vs *prudent* and *deliberate* vs *inadvertent*

🏛️ Learn more in the Library

And that's it for today! You will find this and the attached resources in the Refactoring Library.

See you next week! 👋

Sincerely
Luca



29 Likes · 4 Restacks

Comments



Write a comment...

© 2024 Refactoring ETS · Privacy · Terms · Collection notice
Substack is the home for great culture